

Git and GitHub 101

Self-Study Guide for Python Students

Audience: Students working with Python modules (.py) and Jupyter notebooks (.ipynb) in Cursor, on Windows or macOS.

Package management: uv

Learning objectives

By the end of this guide, you should be able to:

- Explain what Git is and why developers use it.
- Understand the difference between a file, a Git repository, a commit, and a remote repository.
- Use ``git status``, ``git add``, and ``git commit``.
- Explain what GitHub is and how it differs from Git.
- Clone an existing repository with ``git clone``.
- Explain, at a high level, what ``git pull`` and ``git push`` do.
- Recognise the basic workflow you will use in your Python course.

****Important:**** Git tracks your project files and their history. It does not replace ``uv``, your Python environment, or your editor.

Part 1 — What is Git?

1.1 The problem Git solves

Imagine you are working on a Python project. You might have:

```
analysis.py
data_cleaning.py
model.ipynb
README.md
```

Over several days you make changes:

- You fix a bug.

- You add a new function.
- You experiment with a different approach.
- You accidentally break something.
- You want to know what the code looked like yesterday.

Without version control, people often create files such as:

```
analysis.py
analysis_v2.py
analysis_final.py
analysis_final_really_final.py
analysis_final_really_final_2.py
```

This becomes difficult to manage.

Git is a version control system. It records the history of changes to a project so that you can understand, compare, and recover versions of your work.

A useful mental model is:

```
**Git gives your project a history of snapshots.**
```

1.2 A brief history

Git was created in 2005 by Linus Torvalds, the creator of Linux.

It was designed to be:

- fast,
- distributed,
- reliable,
- suitable for large software projects.

Today Git is one of the standard tools used for software development.

1.3 Git is not the same thing as a backup

A Git repository usually contains a history of your project, but Git is not a substitute for a complete backup strategy.

For example, if your laptop is destroyed and your only copy of the repository was on that laptop, Git cannot magically recover it.

This is one reason remote Git hosting services such as GitHub are useful.

1.4 Git repositories

A **Git repository**, often shortened to **repo**, is a folder whose contents are being tracked by Git.

A repository contains your project files plus Git's internal information about the project's history.

When Git is initialized in a folder, Git creates a hidden directory called:

`.git`

You normally should **not edit or delete `.git` manually**.

1.5 Git and Python projects

A typical course repository might look like:

```
my-python-project/
├──
├── README.md
├── pyproject.toml
├── uv.lock
├── src/
│   ├── my_project/
│   │   └── analysis.py
│   └──
├── notebooks/
│   └── exploration.ipynb
```

Git can track all of these files.

Git does not care whether a file is Python, Markdown, JSON, or a Jupyter notebook. It records changes to files.

Git and `uv` have different jobs

It is important not to confuse these tools:

Tool	Main job
Git	Version control
GitHub	Hosting and collaboration around Git repositories
`uv`	Python project/package/environment management
Cursor	Code editor / development environment

Tool	Main job
Jupyter	Interactive notebook environment

For example:

```
uv add pandas
```

changes your Python project's dependencies.

Git can then record that project change:

```
git status
git add pyproject.toml uv.lock
git commit -m "Add pandas dependency"
```

The tools work together, but they solve different problems.

Exercise 1 — Think about version control

Answer these questions before continuing.

1. What problem does Git solve?
2. Why might `analysis_final.py` and `analysis_final_v2.py` become difficult to manage?
3. What is a Git repository?
4. What is the purpose of the `.git` directory?
5. What is the difference between Git and `uv`?

Suggested answers

1. Git records the history of changes to files and helps us manage different versions of a project.
2. It becomes unclear which version is correct and there is no reliable history of how changes were made.
3. A folder containing a project and its Git history.
4. It stores Git's internal repository information and history.
5. Git manages version history; `uv` manages Python dependencies and environments.

Part 2 — Basic Git commands

The three commands to learn first are:

```
git status
git add
git commit
```

A simple mental model is:



`git status` lets you inspect what is happening at each stage.

2.1 `git status`

Run:

```
git status
```

This asks Git:

```
"What is the current state of my repository?"
```

It can tell you things such as:

- which branch you are on,
- which files have changed,
- which files are new,
- which changes are staged,
- whether your working tree is clean.

For example, you might see:

```
On branch main

Changes not staged for commit:
  modified:   analysis.py
```

This means the file has changed, but those changes have not yet been staged for the next commit.

2.2 `git add`

Suppose you edit:

```
analysis.py
```

Git notices that the file has changed.

To tell Git that you want the current changes included in your next commit:

```
git add analysis.py
```

You can then run:

```
git status
```

You may see something like:

```
Changes to be committed:
  modified:   analysis.py
```

The important idea is:

```
`git add` does not create a commit.
```

It places the selected changes into the **staging area**.

2.3 What is the staging area?

The staging area is a preparation area.

It lets you decide exactly which changes should be included in your next commit.

For example, suppose you changed:

```
analysis.py
README.md
notes.txt
```

You might want to commit only the changes to the Python code:

```
git add analysis.py
```

The changes to `README.md` and `notes.txt` are not included in that commit.

This gives you control over what each commit represents.

2.4 `git commit`

Once the changes you want are staged, create a commit:

```
git commit -m "Fix data cleaning function"
```

A **commit** is a recorded point in your project's Git history.

Think of it as a named checkpoint.

A good commit message tells another person what changed.

Good examples:

```
Add customer data cleaning function
Fix missing-value handling
Add exploratory analysis notebook
Update project dependencies
```

Less useful:

```
stuff
changes
update
fixed things
```

2.5 The basic workflow

A very common sequence is:

```
git status
git add analysis.py
git status
git commit -m "Fix analysis function"
git status
```

In plain English:

1. Check what changed.
2. Stage the changes.
3. Check what is staged.
4. Save those staged changes as a commit.
5. Check that everything is as expected.

2.6 `git add .`

You will often see:

```
git add .
```

The `.` means the current directory.

This asks Git to stage changes throughout the current directory and its relevant subdirectories.

It is convenient, but you should understand what you are staging before committing.

A good habit is:

```
git status
git add .
git status
git commit -m "Describe the changes"
```

The second `git status` gives you a chance to check what will be committed.

Exercise 2 — Create your first repository

Create a practice folder.

macOS / Linux terminal

```
mkdir git-practice
cd git-practice
```

Create a file:

```
echo "print('Hello Git')" > hello.py
```

Windows PowerShell

```
mkdir git-practice
cd git-practice
```

Create a file:

```
"print('Hello Git')" | Out-File hello.py
```

You can also simply create `hello.py` using Cursor.

Now initialize Git:

```
git init
```

Check the status:

```
git status
```

You should see `hello.py` as an untracked file.

Stage it:

```
git add hello.py
```

Check again:

```
git status
```

Now commit:

```
git commit -m "Add hello Python script"
```

Finally:

```
git status
```

Your repository should now have a clean working tree.

Exercise 3 — Make a second commit

Open `hello.py` in Cursor and change it to something like:

```
print("Hello Git")  
print("I am learning version control")
```

Then run:

```
git status
```

Stage and commit the change:

```
git add hello.py  
git commit -m "Add second message to hello script"
```

Check the status again:

```
git status
```

Think about this

You now have two commits.

The important idea is that Git has a history:

```
Commit 1
  ↓
Commit 2
```

If you continue working, you can create more checkpoints:

```
Commit 1 → Commit 2 → Commit 3 → Commit 4
```

Part 3 — What is GitHub?

3.1 GitHub is not Git

This distinction is extremely important.

Git is the version control system.

GitHub is an online platform that hosts Git repositories and provides tools for collaboration.

A useful analogy:

Git is the version-control technology; GitHub is one service built around Git.

You can use Git without GitHub.

For example, you can create a repository on your laptop:

```
git init
```

and make commits without ever connecting it to the internet.

3.2 What does GitHub provide?

GitHub provides much more than file storage.

It can provide:

- remote Git repositories,

- collaboration,
- pull requests,
- code review,
- issue tracking,
- project management features,
- documentation,
- access controls,
- automation and CI/CD features.

For your course, the most important concept initially is:

****GitHub gives you a remote place where your Git repository can be hosted and shared.****

3.3 Local versus remote

Imagine your repository exists on your laptop:

```
Your computer
■■■ my-project/
    ■■■ analysis.py
    ■■■ notebook.ipynb
    ■■■ .git/
```

You can also have a copy hosted on GitHub:

The diagram illustrates the state of a local repository (Your computer) and a remote repository (GitHub) after a push. The local repository has a commit 'my-project' and a 'sync' operation. The remote repository has a commit 'my-project' and a 'sync' operation. The diagram shows the flow of data from the local repository to the remote repository.

Commands such as `git push` and `git pull` help synchronize these repositories.

...

3.4 Alternatives to GitHub

GitHub is extremely popular, but it is not the only Git hosting service.

Examples include:

- **GitLab**

- **Bitbucket**
- **Gitea**
- **SourceHut**

Organizations may also host Git repositories on their own infrastructure.

The important concept is not the brand.

The important concept is:

Git is the version control system; GitHub, GitLab, Bitbucket, etc. are services that can host Git repositories and provide collaboration tools.

Exercise 4 — Git or GitHub?

For each statement, decide whether it describes **Git**, **GitHub**, or **both**.

1. Tracks commits.
2. Is an online service.
3. Can be used entirely offline.
4. Hosts remote repositories.
5. Provides pull requests.
6. Creates a project history.
7. Can be used without having a GitHub account.

Suggested answers

1. Git
2. GitHub
3. Git
4. GitHub
5. GitHub
6. Git
7. Git

Part 4 — Why do we run ``git clone``?

One of the most important commands when starting work on an existing course repository is:

```
git clone
```

4.1 The basic idea

Suppose your instructor has a repository on GitHub:

```
https://github.com/example/course-project.git
```

You want to work on it on your own computer.

You can run:

```
git clone https://github.com/example/course-project.git
```

This creates a local copy of the repository.

4.2 What exactly does `git clone` do?

At a high level, `git clone`:

1. Creates a local directory for the repository.
2. Downloads the repository's files.
3. Downloads Git history.
4. Creates the local `.git` repository data.
5. Configures a remote connection, normally called `origin`.
6. Checks out an initial working version of the files.

So cloning is more than simply downloading a ZIP file.

ZIP download versus clone

A ZIP download gives you files.

A Git clone gives you:

- the files,
- the Git repository,
- the commit history,

- information about the remote repository,
- the ability to use Git to pull, commit, and push.

4.3 What is `origin`?

After cloning, Git normally gives the remote repository the name:

```
origin
```

You can inspect this with:

```
git remote -v
```

You may see:

```
origin https://github.com/example/course-project.git (fetch)
origin https://github.com/example/course-project.git (push)
```

`origin` is simply a convenient name.

It is not a special type of server.

You could technically call a remote something else, but `origin` is the standard name created by `git clone`.

4.4 Cloning a course repository

Suppose your instructor gives you:

```
https://github.com/example/python-course.git
```

You might run:

```
git clone https://github.com/example/python-course.git
```

Then:

```
cd python-course
```

You can open that folder in Cursor.

From there, your course may ask you to work with:

```
.py files
.ipynb notebooks
```

```
pyproject.toml
uv.lock
```

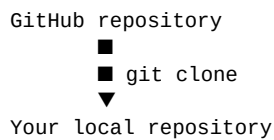
The repository is now on your computer and Git is tracking its history.

4.5 Cloning does not create your own independent Git universe

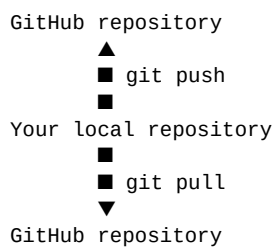
This is a subtle but important point.

After cloning, you have your own local Git repository, but it is connected to the remote repository.

Conceptually:



Later:



The exact mechanics of branches, fetching, merging, and rebasing are more advanced topics and will be covered separately.

Exercise 5 — Clone a repository

Ask your instructor for a course repository URL, or use a repository you have permission to access.

Run:

```
git clone <repository-url>
```

Then:

```
cd <repository-folder>
```

Check:

```
git status
```

Then:

```
git remote -v
```

Answer:

1. What files appeared on your computer?
2. What does `git status` say?
3. What is the name of the remote?
4. What URL is associated with `origin`?
5. Can you find the `.git` directory?

Part 5 — `git pull` and `git push`

These commands are about communicating between your local repository and a remote repository such as GitHub.

For this course, you will have a separate guide covering the repository-specific workflow in more detail.

For now, learn the basic concepts.

5.1 `git push`

Suppose you have made a commit locally:

```
Your computer  
Commit A  
↓  
Commit B
```

But GitHub does not yet know about Commit B.

`git push` sends your local commits to the remote repository.

For example:

```
git push
```

or, when you need to specify the remote and branch:

```
git push origin main
```

Conceptually:

```

Your computer                                GitHub
Commit A → Commit B  ██████████ Commit A → Commit B
                        git push

```

5.2 Important: `git push` does not automatically save uncommitted work

Suppose you edit:

```
analysis.py
```

but have not committed it.

Running:

```
git push
```

does not normally upload that uncommitted change as a Git commit.

The normal sequence is:

```
git add analysis.py
git commit -m "Update analysis"
git push
```

This is why it is useful to think in stages:

```

Edit files
↓
git add
↓
Staged changes
↓
git commit
↓
Local Git history
↓
git push
↓
Remote Git repository

```

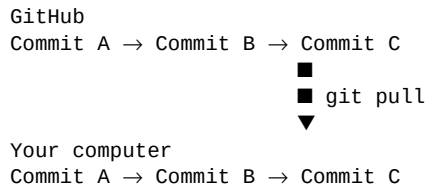
5.3 `git pull`

Now imagine someone else has made a change to the remote repository.

Your local repository may be behind.

`git pull` is used to bring changes from the remote repository into your local repository.

Conceptually:



A simplified description is:

`git pull` gets remote changes and integrates them into your current local branch.

The precise mechanics involve operations such as fetching and integrating changes. You will learn more about this in the separate course guide.

5.4 Why might you need `git pull`?

Imagine your instructor updates the course repository.

Before the update:

```
GitHub:
A → B

Your computer:
A → B
```

The instructor adds Commit C:

```
GitHub:
A → B → C

Your computer:
A → B
```

Your local repository is now behind.

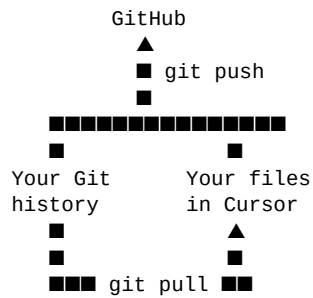
Running:

```
git pull
```

updates your local repository.

5.5 A typical workflow

A simplified workflow might look like:



In practice, you will frequently use:

```
git status
```

to understand what is happening.

A complete beginner workflow

Here is a useful sequence to remember.

Starting work on an existing repository

Usually:

```
git clone <repository-url>
cd <repository-folder>
```

Then open the folder in Cursor.

Before making important changes:

```
git status
```

While working

Edit your `.py` modules or `.ipynb` notebooks.

If you add a dependency with uv, for example:

```
uv add pandas
```

Git may then report changes to files such as:

```
pyproject.toml
uv.lock
```

Check:

```
git status
```

Saving your work to Git history

```
git add .
git status
git commit -m "Add data analysis"
```

Sharing your commits with the remote repository

```
git push
```

Getting changes made elsewhere

```
git pull
```

Your course-specific guide will explain when and how you should use pull and push for the course repository.

Understanding the four important states

A useful way to understand Git is to distinguish four places/states:

1. Working directory
Your actual files

■ git add
▼

2. Staging area
Changes selected for the next commit

■ git commit



3. Local repository
Your Git commit history

■ git push



4. Remote repository
Repository hosted on GitHub

And in the other direction:

Remote repository



■ git pull



Local repository

This model explains a lot of Git behaviour.

Common beginner misunderstandings

"I ran `git add`, so my work is saved."

Not exactly.

`git add` stages changes.

You still need:

`git commit`

to create a commit.

"I committed my work, so it's on GitHub."

Not necessarily.

A commit can exist only on your local machine.

You normally need:

`git push`

to send your commits to the remote repository.

"Git and GitHub are the same thing."

No.

Git is version-control software.

GitHub is a service that hosts Git repositories and provides collaboration features.

"Cloning is the same as downloading a ZIP."

No.

A clone includes Git repository information and history, allowing you to continue using Git with the repository.

"Git stores my Python environment."

Not exactly.

Your Python environment is managed separately. In this course, uv is responsible for Python project and dependency management.

Git can track files such as:

```
pyproject.toml
uv.lock
```

which describe the project's environment/dependencies.

Working in Cursor

Cursor is your development environment; Git operates on the files in your project folder.

A typical workflow is:

1. Clone the repository.
2. Open the cloned folder in Cursor.
3. Edit a .py file or .ipynb notebook.
4. Use the terminal or Cursor's Git interface to inspect changes.
5. Stage changes.
6. Commit changes.
7. Push when appropriate.

You may see Git controls inside Cursor, but it is important to understand the underlying Git commands.

If Cursor provides a button corresponding to **Commit**, understand that it is performing the same fundamental Git operation as:

```
git commit
```

Learning the command-line concepts makes it much easier to understand what Cursor is doing.

Windows and Mac notes

The Git commands themselves are generally the same on Windows and macOS.

For example, these commands work on both:

```
git status
git add .
git commit -m "My changes"
git pull
git push
```

The main differences are usually:

- how Git is installed,
- which terminal application you use,
- shell-specific commands such as `mkdir`, `echo`, or file-manipulation commands.

For course work, you can normally use the integrated terminal in Cursor.

On macOS, the terminal commonly uses a Unix-like shell.

On Windows, Cursor may use PowerShell, Command Prompt, or another shell.

The Git commands themselves remain the same.

Mini-project: Put it all together

Complete this exercise from start to finish.

Step 1 — Clone

Clone the course repository:

```
git clone <course-repository-url>
```

Enter the directory:

```
cd <repository-folder>
```

Step 2 — Inspect

Run:

```
git status
```

Then:

```
git remote -v
```

Step 3 — Make a change

Create or edit a small Python file:

```
print("I am learning Git")
```

Alternatively, make a small, meaningful change to a course notebook.

Step 4 — Inspect

```
git status
```

Ask yourself:

What does Git say has changed?

Step 5 — Stage

```
git add <your-file>
```

Then:

```
git status
```

Ask:

What is different now?

Step 6 — Commit

```
git commit -m "Add Git practice example"
```

Then:

```
git status
```

Step 7 — Push

If your course workflow permits you to push to the repository:

```
git push
```

Otherwise, stop after the commit and follow your instructor's repository instructions.

Final knowledge check

Try answering these without looking back.

1. What is Git?

2. What is a Git repository?
3. What does ``git status`` do?
4. What does ``git add`` do?
5. What does ``git commit`` do?
6. What is the staging area?
7. What is GitHub?
8. What is the difference between Git and GitHub?
9. Name two alternatives to GitHub.
10. What does ``git clone`` do?
11. What is ``origin``?
12. What does ``git push`` do?
13. What does ``git pull`` do?
14. Does ``git push`` automatically commit uncommitted changes?
15. Does Git replace ``uv``?

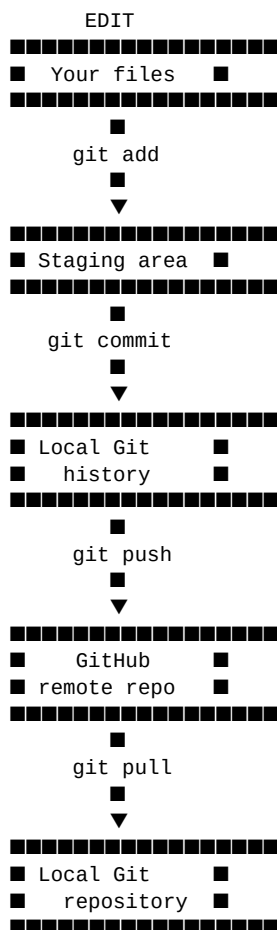
Quick reference

Command	Purpose
<code>`git status`</code>	Show the current state of the repository
<code>`git add <file>`</code>	Stage changes to a file
<code>`git add .`</code>	Stage changes under the current directory

Command	Purpose
<code>`git commit -m "message"``</code>	Create a commit from staged changes
<code>`git clone <url>`</code>	Create a local copy of a remote Git repository
<code>`git remote -v`</code>	Show configured remote repositories
<code>`git pull`</code>	Bring remote changes into the current local branch
<code>`git push`</code>	Send local commits to the remote repository
<code>`git init`</code>	Create a new Git repository in a directory

The most important mental model

If you remember only one diagram from this guide, remember this:



The core workflow is:

```

git status
git add .
git commit -m "Describe what changed"
git push

```

And when you need to receive changes from the remote repository:

```
git pull
```

Remember: `git add` prepares changes, `git commit` records them locally, `git push` sends commits to the remote, and `git pull` brings remote changes into your local repository.